

GROUP C - n8n, Blockchain & DevOps Guide

M5: n8n Workflows | M6: Blockchain + DevOps

What This Guide Covers

- Complete project overview and your team's exact role
- All topics to learn with free online resource links
- Step-by-step environment & tool installation guide
- Google Antigravity (AGY) usage with example prompts
- Week-by-week 16-week build plan with deliverables
- Integration contracts with other groups
- Testing, common mistakes, and daily workflow

Primary Development Tool: Google Antigravity (AGY) IDE

Team Guide: Group C (n8n + Blockchain + DevOps)

AI-Enabled Learning & Competency Gap Platform for India's Official Statistical System (SIH 2026)

1. PROJECT OVERVIEW

The AI-Enabled Learning & Competency Gap Platform is a transformative initiative designed specifically for India's Official Statistical System (SIH 2026). Its primary objective is to dynamically identify the competency gaps of official statisticians and provide them with personalized, AI-driven learning pathways, curated resources, and immutable certifications upon successful completion of assessments.

This platform seamlessly integrates modern generative AI for content curation, robust backend infrastructure for data management, seamless frontend experiences, and immutable blockchain technology for verifiable credentials.

Group C's Role (Infrastructure, Automation, and Blockchain)

Group C is the backbone of the entire system. While other teams focus on AI models, user interfaces, or core API endpoints, Group C ensures that everything runs smoothly, automatically, and verifiably.

- **Infrastructure (DevOps):** You are responsible for the entire deployment stack. Every component--the database, AI services, message queues, and user interfaces--runs inside a Dockerized environment managed by Group C. You configure the CI/CD pipelines to ensure code changes are automatically tested and deployed. You set up monitoring (Grafana, Loki) so the team knows exactly what is happening in the system at all times.
- **Automation (n8n):** The platform relies on asynchronous, event-driven processes. When a user signs up, when a document needs processing by AI, or when a learning reminder needs sending, these events are orchestrated by n8n workflows. You are the conductors of the system's asynchronous symphony.
- **Blockchain (Web3):** The ultimate deliverable for a user is a Competency Certificate. To prevent fraud and ensure verifiable provenance, these certificates are minted as ERC-721 Non-Fungible Tokens (NFTs) on a blockchain testnet. Additionally, critical system events are anchored to the blockchain for tamper-proof auditing.

2. MEMBER RESPONSIBILITIES

Group C consists of two members: M5 and M6. While you collaborate closely, your primary domains are distinct.

Member 5 (M5): The Automation Architect (n8n)

M5 is responsible for all event-driven orchestration and workflow automation using n8n.

- **Workflow Development:** Design, implement, and maintain all 13 core n8n workflows (WF-01 to WF-13).
- **Security:** Implement webhook security using HMAC verification for all incoming requests to n8n to ensure only authorized services can trigger workflows.
- **Integration:** Use HTTP Request nodes to communicate with the FastAPI backend, LLM services, and external APIs.
- **Custom Logic:** Write JavaScript in Code nodes for data transformation and custom processing within workflows.
- **State Management:** Utilize Redis for temporary state storage and rate limiting within workflows.
- **Error Handling:** Implement robust error-handling sub-workflows to manage API failures, timeouts, and retries.
- **Environment Management:** Manage n8n credentials securely across DEV, QA, and PROD environments.

Member 6 (M6): The Blockchain & DevOps Engineer

M6 owns the blockchain integration and the entire infrastructure stack.

Smart Contracts: Write, test, and deploy Solidity smart contracts (CompetencyCertificate.sol and AuditAnchor.sol) using Hardhat.

- Web3 Integration: Develop a Python/Web3.py service to interact with the smart contracts (minting certificates, anchoring logs).
 - IPFS Storage: Integrate with Pinata API to store certificate metadata (JSON) immutably on IPFS.
 - Infrastructure: Architect and maintain the docker-compose.yml defining the full stack (PostgreSQL, pgvector, Redis, MinIO, n8n, Grafana, Loki, Prometheus, FastAPI, Next.js).
 - CI/CD: Create and maintain GitHub Actions workflows for automated linting, testing, building, and Docker image pushing.
 - Observability: Set up Grafana dashboards and Loki log aggregation to monitor system health and AI generation metrics.
 - Object Storage: Configure MinIO for S3-compatible document storage, including bucket policies and presigned URLs.
-

3. TOPICS TO LEARN (with Free URLs)

To succeed, you must master the following technologies. Spend the first two weeks heavily focused on these resources.

M5 Topics (Automation & n8n)

1. n8n Fundamentals: Understand nodes, connections, trigger vs. action nodes, and execution data.
 - n8n Official Documentation
 - n8n Beginners Course
2. Webhook Concepts & Security: Learn how webhooks work and how to secure them using HMAC.
 - What is a Webhook?
 - Securing Webhooks with HMAC
3. HTTP and REST APIs: Deep dive into HTTP methods, status codes, headers, and JSON payloads.
 - MDN Web Docs: HTTP
4. JavaScript Basics (for Code Nodes): You need JS for data manipulation in n8n.
 - JavaScript.info
5. Cron Expressions: For scheduling recurring workflows.
 - Crontab Guru

M6 Topics (Blockchain & DevOps)

1. Solidity Basics: The language for writing smart contracts.
 - Solidity Documentation
 - CryptoZombies (Interactive Tutorial)
 2. Hardhat: The development environment for Ethereum software.
 - Hardhat Getting Started
 3. OpenZeppelin: Secure smart contract libraries (ERC-721).
 - OpenZeppelin Docs
 4. Web3.py: Python library for interacting with Ethereum.
 - Web3.py Documentation
 5. IPFS and Pinata: Decentralized storage for NFTs.
 - Pinata Documentation
 6. Docker and Docker Compose: Containerization.
 - Docker Get Started
 7. GitHub Actions: CI/CD automation.
 - GitHub Actions Documentation
 8. Grafana & Prometheus: Monitoring.
 - Grafana Fundamentals
-

4. ENVIRONMENT SETUP (Step-by-Step)

Follow these steps precisely to set up your local development environment.

Step 1: Docker Desktop Install (M5 & M6)

1. Download Docker Desktop for Windows from docker.com.
2. Install and ensure WSL 2 (Windows Subsystem for Linux) integration is enabled in settings.

Step 2: n8n via Docker (M5)

Run n8n locally to start building workflows immediately.

```
docker volume create n8n_data
docker run -it --rm \
  --name n8n \
  -p 5678:5678 \
  -v n8n_data:/home/node/.n8n \
  n8nio/n8n
```

Open <http://localhost:5678> in your browser and set up your local admin account.

Step 3: Node.js + Hardhat Setup (M6)

1. Install Node.js (LTS version).
2. Create a new directory blockchain and initialize Hardhat:

```
mkdir blockchain && cd blockchain
npm init -y
npm install --save-dev hardhat
npx hardhat
# Select "Create a JavaScript project"
npm install @openzeppelin/contracts dotenv
```

Step 4: MetaMask & Polygon Amoy Testnet (M6)

1. Install MetaMask extension in your browser.
2. Add Polygon Amoy Testnet to MetaMask:
 - Network Name: Polygon Amoy Testnet
 - New RPC URL: <https://rpc-amoy.polygon.technology/>
 - Chain ID: 80002
 - Currency Symbol: MATIC
3. Get test MATIC from a Polygon Faucet.

Step 5: Pinata Account Setup (M6)

1. Go to [Pinata.cloud](https://pinata.cloud) and create a free account.
2. Generate an API Key and API Secret. Save these securely.

Step 6: Python + Web3.py Install (M6)

1. Install Python 3.10+.
2. Install Web3.py in your Python environment:

```
pip install web3 python-dotenv
```

Step 7: Antigravity IDE (M5 & M6)

Antigravity (AGY) is your AI-assisted IDE. Open your project folder (C:\Users\user\SIH) in Antigravity. It will assist you in generating code, workflows, and configurations.

Step 8: Environment Variables

Create a .env file in the root of your respective directories. NEVER commit this to GitHub.

M5 (n8n .env):

```
N8N_ENCRYPTION_KEY=your_secure_random_key
WEBHOOK_HMAC_SECRET=super_secret_hmac_key
API_INTERNAL_URL=http://fastapi:8000
```

M6 (Blockchain .env):

```
PRIVATE_KEY=your_metamask_private_key_do_not_share
AM0Y_RPC_URL=https://rpc-amoy.polygon.technology/
PINATA_API_KEY=your_pinata_key
PINATA_SECRET_KEY=your_pinata_secret
```

5. ANTIGRAVITY USAGE GUIDE

Google Antigravity (AGY) is an agentic IDE that acts as your AI pair programmer. You can prompt it to generate complex configurations and boilerplate code.

What Antigravity CAN do:

- Write complex Solidity smart contracts.
- Generate complete Docker Compose files and GitHub Actions YAMLS.
- Write Python Web3 integration scripts.
- Generate JSON structures representing n8n workflows (which you can then import into the n8n GUI).
- Troubleshoot error messages and logs.

What Antigravity CANNOT do:

- It cannot click buttons inside the n8n graphical interface. You must import the JSON it generates.
- It cannot securely sign transactions using your MetaMask private key (never give AGY your real private key).
- It cannot automatically configure your cloud provider accounts.

Example Prompts for M5 (n8n):

"Generate a JSON definition for an n8n workflow. The workflow should start with a Webhook trigger. It needs to verify an HMAC SHA256 signature in the headers using a crypto node. If valid, it should make a POST HTTP request to http://fastapi:8000/api/process. If the HTTP request fails, route to an error handling sub-workflow."

Example Prompts for M6 (Blockchain & DevOps):

"Write a Solidity ^0.8.20 smart contract named CompetencyCertificate. It should inherit from OpenZeppelin ERC721URIStorage. Only the contract owner should be able to mint. The mint function should take a wallet address and a tokenURI (IPFS link). Include events for minting."

"Create a GitHub Actions workflow YAML for a Python FastAPI project. It needs to trigger on push to main, run flake8 for linting, run pytest, and if tests pass, build a Docker image and push it to GitHub Container Registry (ghcr.io)."

"Generate a docker-compose.yml for a local dev environment that includes PostgreSQL 15 with the pgvector extension, Redis, and MinIO. Ensure MinIO has persistent volumes and set the default bucket name to 'documents'."

6. COMPLETE FOLDER STRUCTURE

Maintain this strict folder structure to ensure consistency across the project.

```
C:\Users\user\SIH\
??? .github/
?   ??? workflows/
?     ??? ci-cd.yml           # GitHub Actions pipeline
??? blockchain/
?   ??? contracts/
? ?   ??? CompetencyCertificate.sol
? ?   ??? AuditAnchor.sol
?   ??? scripts/
? ?   ??? deploy.js          # Hardhat deployment script
?   ??? test/
? ?   ??? CompetencyCertificate.test.js
?   ??? hardhat.config.js
?   ??? package.json
??? infra/
?   ??? docker-compose.yml    # Full stack definition
?   ??? grafana/
? ?   ??? provisioning/
? ?   ??? dashboards/
?   ??? prometheus/
?     ??? prometheus.yml
??? n8n/
?   ??? workflows/
? ?   ??? WF-01_Catalog_Sync.json
? ?   ??? WF-02_Profile_Sync.json
? ?   ??? WF-03_Baseline_Assessment.json
? ?   ??? WF-04_Recommendation.json
? ?   ??? WF-05_Document_Ingestion.json
? ?   ??? WF-06_Quiz_Generation.json
```

7. ALL 13 n8n WORKFLOWS (Detailed)

M5 must build and maintain these workflows. To import a workflow into n8n, you can copy the JSON generated by Antigravity, go to your n8n workspace, click "Add Workflow", and simply press Ctrl+V (or Cmd+V) to paste the JSON structure directly onto the canvas.

Webhook Security Standard (Applies to all Webhook triggers)

Every Webhook node must be followed immediately by an HMAC Verification process.

1. Extract the X-Hub-Signature-256 header from the request.
2. Use a Crypto node to generate an HMAC SHA256 signature of the raw body using the WEBHOOK_HMAC_SECRET.
3. Use an IF node to compare the received signature with the calculated signature. If they don't match, return a 401 Unauthorized HTTP response and stop execution.

WF-01: Catalog Sync

- Trigger Type: Cron (Nightly at 02:00 AM)
- Description: Pulls new statistical guidelines and manuals from external government portals.
- Nodes:
 1. Cron Trigger (0 2 *)
 2. HTTP Request (GET external RSS/API)
 3. Item Lists (Split out items)
 4. HTTP Request (POST to Group B's /api/ingest endpoint)
- Error Handling: If external API is down, route to WF-12 Error Recovery and notify admins via email.

WF-02: Profile Sync

- Trigger Type: Webhook (POST /webhook/profile-sync)
- Description: Triggered when HR updates a user's role in the legacy system. Updates the competency baseline requirements.
- Nodes:
 1. Webhook (POST)
 2. HMAC Verification Sub-workflow
 3. Set (Map incoming HR data to platform schema)
 4. HTTP Request (PATCH to internal /api/users/{id}/profile)
- Test Payload: {"user_id": "U123", "new_role": "Senior Statistician", "effective_date": "2026-09-01"}

WF-03: Baseline Assessment

- Trigger Type: Webhook (POST /webhook/assessment-complete)
- Description: Triggered when a user finishes their initial diagnostic test. Orchestrates the scoring and gap analysis.
- Nodes:
 1. Webhook (POST)
 2. HMAC Verification
 3. HTTP Request (POST to AI Service /api/ai/analyze-gaps)
 4. IF Node (Check if gap severity is High or Low)
 5. HTTP Request (POST back to Core Backend to save gap profile)

WF-04: Recommendation

- Trigger Type: Message Queue / Redis Trigger
- Description: After a gap is identified, this workflow triggers the AI to generate a personalized learning pathway.
- Nodes:
 1. Trigger (Redis PubSub or Polling)
 2. HTTP Request (POST to AI Service /api/ai/recommend-pathway)
 3. Code Node (Format the recommendations into UI-friendly JSON)
 4. Postgres Node (Insert directly into recommendations table)

WF-05: Document Ingestion

- Trigger Type: Webhook (POST /webhook/document-uploaded)
- Description: Handles new documents uploaded to MinIO. Triggers OCR, chunking, and vector embedding.
- Nodes:
 1. Webhook (POST) - Payload contains minio_object_path
 2. HTTP Request (GET document from MinIO)
 3. HTTP Request (POST to AI Service /api/ai/process-document)
 4. Sub-workflow (Call WF-13 to anchor document hash on blockchain for provenance)

WF-06: Quiz Generation

- Trigger Type: Webhook (POST /webhook/generate-quiz)
- Description: User requests a quiz for a specific topic.
- Nodes:
 1. Webhook (POST)
 2. HTTP Request (POST to AI Service to generate questions)
 3. Code Node (Validate JSON schema of questions)
 4. Webhook Response (Return generated quiz directly to the HTTP response to the frontend)

WF-07: Human Review

- Trigger Type: Webhook (POST /webhook/flag-content)
- Description: If AI generates content with low confidence, or a user reports an error, route to human review.

Nodes:

1. Webhook (POST)
2. HTTP Request (POST to Slack/Teams channel with alert)
3. Postgres Node (Update content status to PENDING_REVIEW)

WF-08: Learning Reminder

- Trigger Type: Cron (Daily at 09:00 AM)
- Description: Checks for users who haven't logged in for 7 days or have pending assessments, and sends email reminders.
- Nodes:
 1. Cron Trigger
 2. Postgres Node (SELECT users where last_login < NOW() - INTERVAL '7 days')
 3. Item Lists (Loop over users)
 4. Send Email Node (SMTP integration to send polite reminder)

WF-09: Outcome Telemetry

- Trigger Type: Webhook (POST /webhook/module-complete)
- Description: Records learning interactions (time spent, clicks, answers) into the time-series database.
- Nodes:
 1. Webhook (POST)
 2. Code Node (Format data for Loki/Prometheus)
 3. HTTP Request (POST to Grafana Loki push API)

WF-10: Reassessment

- Trigger Type: Cron (Weekly)
- Description: Automatically assigns reassessment quizzes to users 30 days after they complete a module to check retention.
- Nodes:
 1. Cron Trigger
 2. Postgres Node (Find eligible users)
 3. HTTP Request (POST to Backend to schedule reassessment)

WF-11: Model Evaluation

- Trigger Type: Cron (Monthly)
- Description: Runs an automated evaluation of the LLM's performance against a gold-standard dataset.
- Nodes:
 1. Cron Trigger
 2. HTTP Request (Trigger internal evaluation script)
 3. HTTP Request (Save results to evaluation metrics table)

WF-12: Error Recovery

- Trigger Type: Error Trigger (Sub-workflow)
- Description: A global error handler called by other workflows when they fail.
- Nodes:
 1. Error Trigger
 2. Code Node (Extract error message and workflow ID)
 3. HTTP Request (POST to Slack/Teams for immediate developer alert)
 4. Postgres Node (Log error into system_errors table for auditing)

WF-13: Blockchain Audit Anchor

- Trigger Type: Webhook / Sub-workflow
- Description: Takes a SHA256 hash of a critical event (e.g., certificate issuance, document ingestion) and anchors it to the AuditAnchor smart contract.

Nodes:

1. Webhook (Internal only)
2. HTTP Request (POST to Python Web3 Service /api/web3/anchor)
3. Postgres Node (Update record with the blockchain Transaction Hash)

8. BLOCKCHAIN GUIDE

Blockchain is utilized in this platform not for cryptocurrency, but for verifiable credentials and immutable audit trails.

When an official statistician completes a major competency milestone, they are issued a Competency Certificate. By minting this as an ERC-721 NFT on the Polygon Amoy testnet, the certificate becomes cryptographically verifiable. Anyone can prove the statistician earned the certificate without relying on the internal database.

Smart Contract 1: CompetencyCertificate.sol

This contract mints NFTs representing competency certificates.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract CompetencyCertificate is ERC721URIStorage, Ownable {
    uint256 private _nextTokenId;

    event CertificateMinted(address indexed recipient, uint256 indexed tokenId, string tokenURI);

    constructor() ERC721("SIH Competency Certificate", "SIHCC") Ownable(msg.sender) {}

    // Only the platform backend (owner) can mint certificates
    function mintCertificate(address recipient, string memory tokenURI) public onlyOwner returns (uint256) {
        uint256 tokenId = _nextTokenId++;
        _mint(recipient, tokenId);
        _setTokenURI(tokenId, tokenURI);

        emit CertificateMinted(recipient, tokenId, tokenURI);

        return tokenId;
    }
}
```

Smart Contract 2: AuditAnchor.sol

Used to anchor hashes of system logs to prove they haven't been tampered with.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/Ownable.sol";

contract AuditAnchor is Ownable {
    event LogAnchored(bytes32 indexed dataHash, uint256 timestamp, string description);

    constructor() Ownable(msg.sender) {}

    function anchorLog(bytes32 dataHash, string memory description) public onlyOwner {
        emit LogAnchored(dataHash, block.timestamp, description);
    }
}
```

IPFS Metadata Structure

Before minting, the Python service uploads a JSON file to IPFS via Pinata. The resulting IPFS URI is used as the tokenURI.

```
{
  "name": "Advanced Data Imputation Specialist",
  "description": "Awarded for demonstrating mastery in statistical data imputation techniques.",
  "image": "ipfs://QmYourImageHash...",
  "attributes": [
    { "trait_type": "Competency", "value": "Data Imputation" },
    { "trait_type": "Level", "value": "Advanced" },
    { "trait_type": "Issue Date", "value": "2026-09-02" }
  ]
}
```

Web3.py Integration Service (Python snippet)

The Web3 service acts as the bridge between n8n/FastAPI and the blockchain.

```

from web3 import Web3
import json
import os
from dotenv import load_dotenv

load_dotenv()

w3 = Web3(Web3.HTTPProvider(os.getenv("AM0Y_RPC_URL")))
private_key = os.getenv("PRIVATE_KEY")
account = w3.eth.account.from_key(private_key)

# Load ABI and Contract Address
with open("CompetencyCertificate.json") as f:
    contract_json = json.load(f)
contract = w3.eth.contract(address="0xYourContractAddress...", abi=contract_json["abi"])

def mint_nft(recipient_address, ipfs_uri):
    nonce = w3.eth.get_transaction_count(account.address)

    # Build transaction
    tx = contract.functions.mintCertificate(recipient_address, ipfs_uri).build_transaction({
        'chainId': 80002,
        'gas': 2000000,
        'maxFeePerGas': w3.to_wei('2', 'gwei'),
        'maxPriorityFeePerGas': w3.to_wei('1', 'gwei'),
        'nonce': nonce,
    })

    # Sign and send

```

WARNING: NEVER deploy to Ethereum Mainnet or Polygon Mainnet for the SIH demo. It costs real money. Always use the Polygon Amoy Testnet.

9. DOCKER COMPOSE

This is the definitive docker-compose.yml for running the entire local stack. M6 is responsible for maintaining this.

```
version: '3.8'

services:
  # Database with pgvector for AI embeddings
  postgres:
    image: ankane/pgvector:v0.5.1
    environment:
      POSTGRES_USER: sih_user
      POSTGRES_PASSWORD: sih_password
      POSTGRES_DB: sih_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - sih_network

  # Redis for caching, n8n state, and queueing
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    networks:
      - sih_network

  # MinIO for S3-compatible document storage
  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
```

10. GITHUB ACTIONS CI/CD

M6 must set up CI/CD to automatically test and build the Web3 service and Backend services. Create `.github/workflows/ci-cd.yml`:

```

name: CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install flake8 pytest web3

      - name: Lint with flake8
        run: |
          flake8 services/web3-service --count --select=E9,F63,F7,F82 --show-source --statistics

      - name: Run Tests

```

11. GRAFANA DASHBOARDS

Grafana is crucial for visualizing the health of the system during the SIH evaluation. M6 should configure dashboards to track:

1. **API Latency (FastAPI):** Track response times for critical endpoints. If AI generation takes >5 seconds, we need to know.
2. **n8n Workflow Success Rates:** Use n8n's Prometheus metrics exporter to visualize how many workflows succeeded, failed, or are currently active.
3. **AI Token Usage:** Track how many tokens the external LLM is consuming to manage costs.
4. **Blockchain Transactions:** Display the count of Competency Certificates minted and Audit Anchors recorded.
5. **System Health:** CPU and RAM usage of the Docker containers.

12. MINIO SETUP

MinIO provides local, S3-compatible storage. It is used to store PDFs of statistical manuals, user profile pictures, and raw JSON logs.

Configuration Steps:

1. Start MinIO via Docker Compose.
2. Log in to the console at <http://localhost:9001> (user: minioadmin, pass: minioadmin123).
3. Create a bucket named sih-documents.
4. Set the Access Policy of sih-documents to Public (read-only) so the frontend can display PDFs using the object URL.
5. In your FastAPI/n8n settings, use the AWS S3 SDK (boto3 or equivalent) pointing to <http://localhost:9000> instead of AWS.

13. WEEK-BY-WEEK TASKS (16 Weeks to SIH)

Phase 1: Foundation (Weeks 1-4)

- Week 1: Complete environment setup (Docker, MinIO, Postgres). M5 completes n8n tutorials. M6 sets up Hardhat and deploys dummy contract to Amoy.
- Week 2: M5 builds WF-01 and WF-02. M6 writes Web3.py wrapper and test scripts.
- Week 3: Establish CI/CD pipeline. Setup webhook security (HMAC) for n8n.
- Week 4: Finalize Smart Contracts (CompetencyCertificate.sol, AuditAnchor.sol).

Phase 2: Core Integration (Weeks 5-10)

- Week 5: M5 builds WF-03 (Baseline) and WF-04 (Recommendation). Connect to Group B's APIs.
- Week 6: M6 sets up Prometheus and Grafana. Start piping n8n metrics.
- Week 7: M5 builds WF-05 (Document Ingestion) and WF-06 (Quiz Generation).
- Week 8: M6 integrates Pinata for IPFS metadata storage.
- Week 9: Connect Web3 Service to n8n (WF-13 Blockchain Audit).
- Week 10: M5 builds WF-07, WF-08, WF-09.

Phase 3: Refinement & Security (Weeks 11-14)

- Week 11: Implement WF-12 (Error Recovery). Run chaos testing (kill containers, see if system recovers).
- Week 12: Security audit. Ensure no private keys are in code. Verify HMAC on all webhooks.
- Week 13: Build Grafana dashboards for the SIH presentation.
- Week 14: End-to-end testing with mock statistical data.

Phase 4: Freeze & Rehearse (Weeks 15-16)

- Week 15: Code freeze. No new features. Fix bugs only.
- Week 16: Presentation rehearsals. Ensure Docker compose spin-up takes less than 3 minutes on a fresh laptop.

14. INTEGRATION WITH OTHER GROUPS

Group C does not exist in a vacuum. You are the glue.

- With Group B (Backend API): Group B writes the FastAPI endpoints. M5 (n8n) must know the exact JSON schema required by Group B. M5 will frequently use HTTP Request nodes to POST data to Group B, and Group B will fire Webhooks to trigger n8n workflows.
- With Group A (AI/ML): Group A provides the LLM prompts and vector database logic. M5 will orchestrate the data flow (e.g., pulling a document from MinIO and sending it to Group A's processing endpoint).
- With Group D (Frontend): M6 must provide Group D with the ABI and Contract Address of the deployed smart contracts so they can display the user's NFT certificates in the React/Next.js UI.

15. TESTING GUIDE

- n8n Fixtures: Always save sample JSON payloads from webhooks as "Pinned Data" in n8n. This allows you to re-run and test a workflow without needing to trigger the real event over and over.
- Hardhat Tests: Write extensive JavaScript tests in Hardhat to ensure your Solidity contracts work.

```
const { expect } = require("chai");
describe("CompetencyCertificate", function () {
  it("Should mint a new certificate and assign it to owner", async function () {
    const [owner, addr1] = await ethers.getSigners();
    const Cert = await ethers.getContractFactory("CompetencyCertificate");
    const cert = await Cert.deploy();
    await cert.mintCertificate(addr1.address, "ipfs://test");
    expect(await cert.ownerOf(0)).to.equal(addr1.address);
  });
});
```

- Docker Health Checks: Add healthcheck configurations to your docker-compose.yml to ensure services like Postgres are fully ready before n8n starts.

16. SECURITY CHECKLIST

- [] Webhook HMAC: All incoming webhooks to n8n verify an HMAC signature.
- [] No Secrets in Repo: Use .env files. Ensure .env is in .gitignore. Use GitHub Secrets for CI/CD.
- [] Private Keys: Never commit the MetaMask private key used for deploying contracts.
- [] Testnet Only: Verify Hardhat is configured for Polygon Amoy, NOT Mainnet.
- [] Database Passwords: Change default passwords in docker-compose.yml before final deployment.
- [] MinIO Access: Ensure buckets containing PII are private; only public data (like blank certificates) should be public.

17. COMMON MISTAKES TO AVOID

1. Infinite n8n Loops: Be careful when a workflow triggers an API that in turn triggers a webhook back to the same workflow. This will crash your server and consume all resources.
2. Hardcoding URLs: Never hardcode localhost in n8n HTTP nodes if running in Docker. Use the container name (e.g., http://fastapi:8000).
3. Losing the Private Key: If you lose the private key that deployed the smart contract, you lose ownership and cannot mint new certificates. Back it up securely.
4. Ignoring Gas Fees: Even on testnets, transactions require gas (MATIC). Ensure your faucet wallet has enough funds before demo day.

18. DAILY WORKFLOW

1. Pull latest code from main.
2. Run docker-compose up -d to start the infrastructure.
3. Open http://localhost:5678 for n8n.
4. Check Grafana at http://localhost:3000 to ensure no lingering errors from yesterday.
5. Work on assigned tasks (developing workflows or writing contracts).
6. Commit code (export n8n JSONs and commit them to git).
7. Push to branch and create Pull Request.

19. RESOURCES & REFERENCE LINKS

- n8n Best Practices
- OpenZeppelin ERC721 Documentation
-

- Polygon Faucet (Amoy)
 - Docker Compose Networking
 - FastAPI Documentation
-

End of Guide